Equipo

Equipo 2F, conformado por los estudiantes:

- Sergio Andres Hernández Salinas
- Juan Esteban Medina Cardenas
- Juan Pablo Barreto Hernández
- Juan Esteban Santacruz Corredor
- Andres Felipe Ramirez Fajardo
- Cristian Felipe Ramirez Montenegro
- Diego Fernando Mellizo Pedraza
- Julian Santiago Becerra Pulido

Sistema de software

Nombre

GeoPicto

Logo



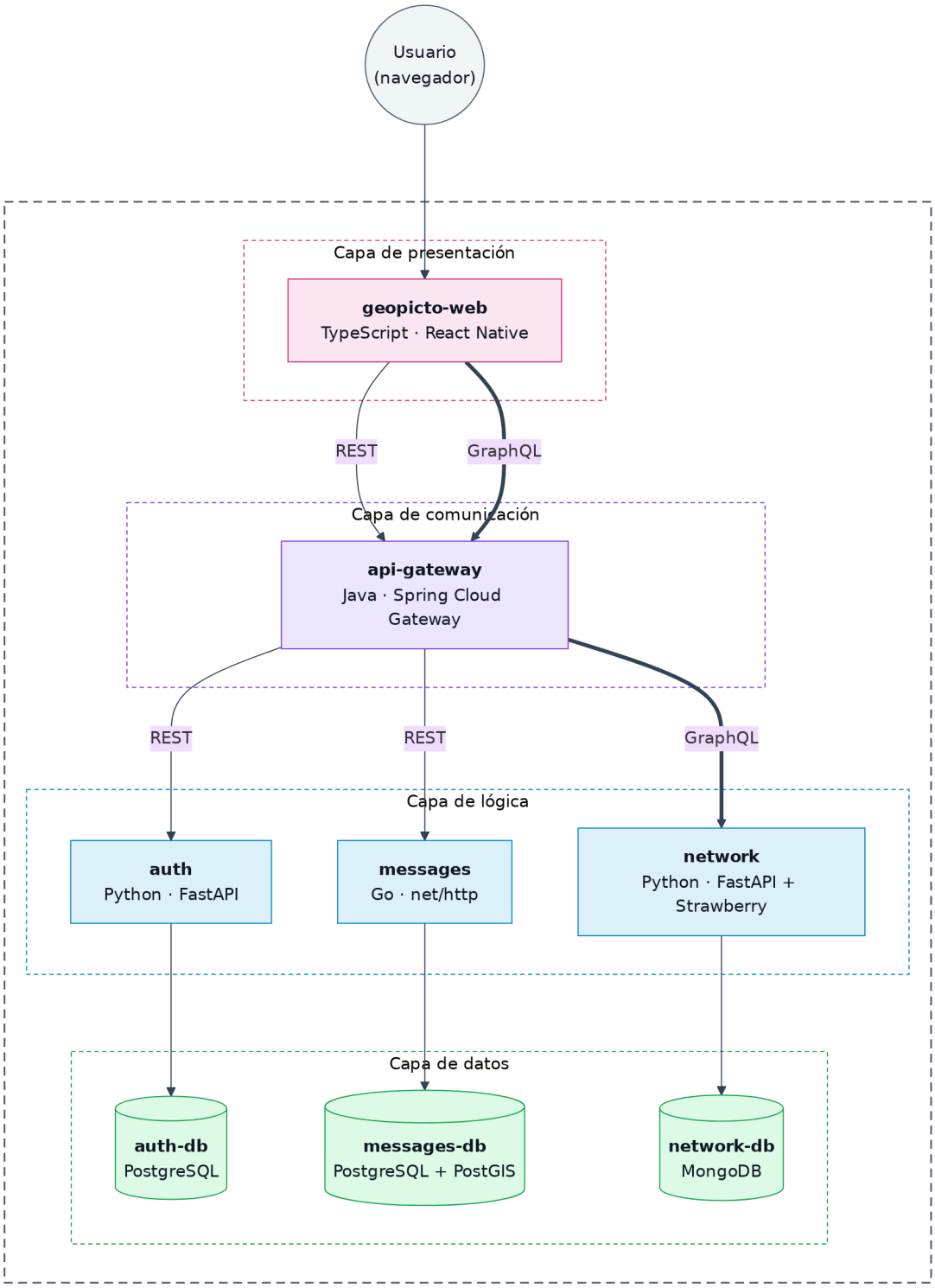
Descripción

Nuestro proyecto lleva por nombre **GeoPicto** y consiste en una red social de mensajes geoespaciales contextuales, en la cual los usuarios pueden crear, descubrir y valorar mensajes asociados a ubicaciones específicas. La aplicación busca integrar las características de una red social con la geolocalización, permitiendo que los usuarios interactúen con contenido creado por otras personas dependiendo de su proximidad.

Estructuras arquitectónicas

Estructura de Componentes y Conectores (C&C)

Vista C&C



Convenciones del diagrama

Elemento gráfico	Significado
Rectángulo	Componente en ejecución (cada uno es un contenedor Docker), con su lenguaje y framework.
Cilindro	Componente de datos, con su motor de base de datos (cada uno en su propio contenedor).
Flecha delgada con etiqueta "REST"	Conector **REST**.
Flecha gruesa con etiqueta "GraphQL"	Conector **GraphQL**.
Flecha sin etiqueta del usuario al frontend	El usuario abre la aplicación web en su navegador.
Flecha sin etiqueta de un servicio a una base de datos	Acceso del servicio de lógica a su propia base de datos.
Sentido de las flechas	De quien inicia la petición hacia quien la atiende. El diagrama se lee de arriba hacia abajo: usuario, presentación, comunicación, lógica y datos.
Recuadro punteado de color	Capa lógica a la que pertenecen los componentes.
Recuadro punteado exterior (gris)	Límite del sistema GeoPicto: todos los componentes que se despliegan con Docker. El usuario queda por fuera.

Estilos arquitectónicos utilizados

1. Cliente-servidor distribuido. El sistema se divide en procesos independientes que se comunican por red. El navegador del usuario es el cliente y el resto de componentes son servidores que corren en contenedores separados. Ningún componente comparte memoria ni sistema de archivos con otro: toda la interacción ocurre a través de conectores REST y GraphQL, y del acceso de cada servicio a su base de datos.

2. Microservicios. La lógica de negocio está repartida en tres servicios pequeños, cada uno con una única responsabilidad del dominio:

- **auth:** identidad y sesiones.
- **messages:** notas geolocalizadas.
- **network:** relaciones de amistad.

Cada servicio se construye, despliega y escala por separado, y está escrito en el lenguaje que mejor se adapta a su problema: Go para las consultas espaciales y Python para auth y network.

Se aplica el patrón **base de datos por servicio**: cada microservicio es dueño exclusivo de su base de datos y ningún otro componente accede a ella directamente. Esto permite usar el motor más adecuado en cada caso:

- PostgreSQL para los usuarios y sesiones (datos estructurados con restricciones de unicidad).
- PostgreSQL con PostGIS para las búsquedas por proximidad.
- MongoDB para las relaciones entre usuarios (documentos flexibles).

3. API Gateway. El `api-gateway` es el **punto único de entrada**: el frontend solo conoce su URL y el gateway enruta cada petición al servicio interno según la ruta (`/api/auth/**` , `/api/messages/**` , `/api/network/graphql`). Además concentra responsabilidades transversales (*gateway offloading*):

- Verifica la firma y la vigencia de los JWT.
- Traduce el token a la cabecera `X-User-ID`.
- Elimina cualquier `X-User-ID` enviado por el cliente.
- Aplica la política CORS para el navegador.

Solo el gateway y el frontend están publicados fuera de la red de Docker. Los servicios y las bases de datos son inaccesibles desde afuera.

4. Arquitectura por capas (n-tier). A nivel de sistema, los componentes se organizan en cuatro capas con dependencias en un solo sentido:

- **presentación:** `geopicto-web`.
- **comunicación:** `api-gateway`.
- **lógica:** `auth`, `messages`, `network`.
- **datos:** `auth-db`, `messages-db`, `network-db`.

Una capa solo invoca a la inmediatamente inferior: la presentación nunca habla con un servicio ni con una base de datos directamente.

Dentro de cada microservicio se repite el estilo por capas: *handler / router* (HTTP) → *service* (reglas de negocio) → *repository* (persistencia).

5. Estilos de interacción REST y GraphQL. La comunicación síncrona usa dos estilos sobre HTTP:

- **REST:** recursos identificados por URL, verbos HTTP y representaciones JSON. Lo usan `auth` y `messages`, cuyas operaciones son acciones simples sobre recursos: registrarse, iniciar sesión, crear, buscar y borrar notas.
- **GraphQL:** un único endpoint con un esquema tipado, donde el cliente declara exactamente qué datos necesita y puede combinar varias consultas en una sola petición. Es la interfaz del servicio `network` y la consume la sección de amigos del frontend: con una sola petición obtiene la lista de amigos y las solicitudes recibidas y enviadas.

Cada servicio expone un único estilo de interfaz, elegido según cómo se consumen sus datos.

6. Autenticación basada en tokens sin estado. El servicio `auth` emite un JWT firmado (HS256) con el id del usuario. El gateway lo verifica localmente con el secreto compartido, sin consultar a `auth` en cada petición. Un *refresh token* rotativo, guardado en la base de datos de `auth`, permite renovar el JWT y revocar sesiones.

Elementos arquitectónicos y relaciones

Componentes

Componente	Tipo	Tecnología	Responsabilidad	Interfaz que expone
`geopicto-web`	Presentación (web front-end)	TypeScript, React Native Web (Expo), MapLibre; servido por nginx	Interfaz de usuario con dos pantallas: registro e inicio de sesión, y mapa con la ubicación del usuario y sus notas cercanas (REST). Desde el mapa se abre la sección de amigos, con la lista de amigos y las solicitudes enviadas y recibidas (GraphQL). Guarda el JWT y lo renueva con el refresh token.	Interfaz gráfica web

Componente	Tipo	Tecnología	Responsabilidad	Interfaz que expone
`api-gateway`	Comunicación / punto de entrada	Java 21, Spring Boot 4, Spring Cloud Gateway (WebFlux), jjwt	Enrutamiento por ruta, verificación de JWT, inyección de `X-User-ID`, CORS.	`/api/auth/**`, `/api/messages/**`, `/api/network/graphql` y `/actuator/health`
`auth`	Lógica	Python 3.13, FastAPI, SQLAlchemy, Argon2, PyJWT	Registro, login, emisión de JWT, refresh tokens rotativos, sesiones revocables, bloqueo por intentos fallidos, perfil.	REST: `/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/profile`, `/auth/logout`, `/auth/sessions` …
`messages`	Lógica	Go 1.27, `net/http`	Publicar notas con coordenadas, buscar notas dentro de un radio (ordenadas por distancia), consultar y borrar las propias.	REST: `POST /messages`, `GET /messages/nearby`, `GET` y `DELETE /messages/{id}`
`network`	Lógica	Python 3.13, FastAPI, Strawberry GraphQL	Solicitudes de amistad (enviar, aceptar, rechazar, cancelar) y lista de amigos.	GraphQL: `POST /graphql`
`auth-db`	Datos (relacional)	PostgreSQL 18	Tablas `users` y `sessions`.	Solo accesible por `auth`
`messages-db`	Datos (relacional)	PostgreSQL 18 + PostGIS 3.6	Tabla `messages` con columna `GEOGRAPHY(Point, 4326)` e índice espacial GiST.	Solo accesible por `messages`
`network-db`	Datos (NoSQL, documental)	MongoDB 8	Colección `friendships` con índice único por pareja de usuarios.	Solo accesible por `network`

Conectores

Conector	Desde → hacia	Detalles
REST	`geopicto-web` → `api-gateway`	Autenticación (`/api/auth/**`) y mensajes (`/api/messages/**`). Cabecera `Authorization: Bearer <JWT>` en las rutas protegidas.
GraphQL	`geopicto-web` → `api-gateway`	Sección de amigos (`/api/network/graphql`). Cabecera `Authorization: Bearer <JWT>`.
REST	`api-gateway` → `auth`	`/api/auth/**` → `/auth/**`. Pública: es donde se obtiene el JWT.
REST	`api-gateway` → `messages`	`/api/messages/**` → `/messages/**`. `GET /nearby` es pública; el resto exige JWT y llega con `X-User-ID`.
GraphQL	`api-gateway` → `network`	`/api/network/graphql` → `/graphql`. Exige JWT y llega con `X-User-ID`. Consultas: `me`, `friends`, `incomingRequests`, `outgoingRequests`, `friendship`. Mutaciones: `sendFriendRequest`, `acceptFriendRequest`, `rejectFriendRequest`, `cancelFriendRequest`, `removeFriend`.

Los dos tipos de conectores basados en HTTP son **REST** y **GraphQL**, y ambos se usan de punta a punta: desde la interfaz web, pasando por el gateway, hasta los servicios de lógica. Además, cada servicio de lógica accede únicamente a su propia base de datos.

Relaciones y restricciones

1. **El frontend solo se relaciona con el gateway.** No conoce las URLs internas de los servicios.
2. **El gateway es el único que puede afirmar la identidad del usuario.** Verifica el JWT, quita la cabecera `Authorization` y agrega `X-User-ID` con el claim `sub` del token. En todas las rutas elimina cualquier `X-User-ID` que envíe el cliente, así que no puede falsificarse.
3. **Los servicios de lógica confían en `X-User-ID`** y no validan tokens. Solo son alcanzables desde la red interna, a través del gateway.
4. **`auth` y `api-gateway` comparten el secreto `JWT_SECRET`.** `auth` lo usa para firmar los tokens y el gateway para verificarlos. El claim `iss` debe ser `geopicto-auth`.
5. **Cada servicio de lógica se relaciona con una sola base de datos, que es exclusivamente suya.** No hay relaciones entre servicios de lógica ni entre bases de datos.
6. **Los servicios se relacionan entre sí solo por el UUID del usuario**, que emite `auth` y que `messages` y `network` guardan como referencia (`sender_id`, `user_a`, `user_b`). No hay claves foráneas entre bases de datos.

Prototipo

Requisitos

- Git
- Docker Desktop (incluye Docker Compose), encendido
- Node.js 18 o superior (opcional, solo para la prueba automática)

Descargar los artefactos

```
git clone https://github.com/jumedinaca/swarch-project.git
cd swarch-project
```

Ejecutar el sistema

No requiere configuración previa. Desde la carpeta `swarch-project`:

```
docker compose up --build -d
```

La primera vez tarda varios minutos porque construye todas las imágenes. Para ver el estado de los contenedores:

```
docker compose ps
```

Los 8 contenedores deben aparecer como `Up (healthy)`.

El secreto con el que se firman los JWT tiene un valor por defecto en `docker-compose.yml`, pensado solo para ejecutar el prototipo en local. Para usar uno propio, basta con copiar `.env.example` a `.env` y definir `JWT_SECRET` (mínimo 32 caracteres).

Qué	URL
Aplicación web	http://localhost:8081
API Gateway	http://localhost:8080
Salud del gateway	http://localhost:8080/actuator/health

Verificar el funcionamiento

Prueba automática de punta a punta. Recorre la app con dos usuarios nuevos pasando por todos los componentes y conectores: registro, login, mensajes (REST), amistades (GraphQL) y CORS.

```
node scripts/smoke-test.mjs
```

Debe terminar con `Todo el sistema funciona.`

Detener el sistema

```
docker compose down        # detiene y elimina los contenedores (conserva los datos)
docker compose down -v     # además borra los volúmenes con los datos
```